

# INSTITUTO TECNOLÓGICO SUPERIOR DEL OCCIDENTE DEL ESTADO DE HIDALGO

## DIVISIÓN DE INGENIERÍA EN SISTEMAS COMPUTACIONALES

---

### Manual Técnico y Documentación de Código Fuente

#### Estación de Diagnóstico Cardiovascular y Calibración Fotométrica de Piel

Sistema Embebido ESP32 Mini, Acelerometría MPU6050, Cancelación Adaptativa de Ruido de Movimiento NLMS, Filtrado Butterworth SOS, Monitoreo Autónomo de Batería Li-Po y Calibración Clínica Individual

---

#### Proyecto:

**Monitor Continuo No Invasivo de Signos Vitales (BPM, Presión Arterial PWA y SpO2) con Cancelación Adaptativa de Ruido de Movimiento y Compensación Melánica Cutánea**

#### Asesores:

- Mtra. Lorena Mendoza Guzmán
- Mtra. Cristy Elizabeth Aguilar Ojeda

#### Alumno / Desarrollador:

- Alejandro Pérez

#### Grupo:

- 8.º Semestre - Grupo A
-

# Contenido

- **1. Módulos de Firmware Embebido (ESP32 Mini)**
  - 1.1 Configuración del Bus I2C Compartido (400 kHz) e Inicialización del Sensor MAX30102
  - 1.2 Programación de Registros Ópticos y Parámetros Biofísicos del Sensor MAX30102
  - 1.3 Configuración y Lectura Inercial Triaxial del Acelerómetro MPU6050 ( $\pm 2g$ )
  - 1.4 Monitoreo de Batería con Divisor Resistivo en ADC1 (GPIO 34) e Histéresis Clínica
  - 1.5 Servidor BLE GATT, Anuncios y Empaquetamiento de la Trama Extendida de 20 Bytes
  - 1.6 Característica GATT de Telemetría de Batería y Banderas de Estado Energético
  - 1.7 Manejador de Comandos de Control Inalámbrico (START / STOP) y Sincronización
  - 1.8 Reconexión Automática y Bucle Asíncrono de Muestreo Sincrónico a 100 Hz
- **2. Módulos de Procesamiento Digital de Señales (DSP)**
  - 2.1 Acondicionamiento de Señal y Filtro Digital IIR Butterworth en Cascada SOS ( `ButterworthFilter` )
  - 2.2 Cancelación Adaptativa de Ruido Inercial por Mínimos Cuadrados Normalizados ( `NLMSFilter` )
  - 2.3 Máquina de Estados de Movimiento Fisiológico y Detección de Contacto Cutáneo
  - 2.4 Clasificación Automática de Fototipo y Tono de Piel por Nivel Continuo DCraw ( `MainWindow._update_auto_skin_tone` )
  - 2.5 Compensación Fotométrica de Melanina y Corrección Espectral de SpO2 por Cociente de Ratios ( `MainWindow.calculate_spo2` )
- **3. Módulos de la Aplicación de Escritorio (Python / PyQt6)**
  - 3.1 Declaración de Dependencias, Librerías del Sistema y Constantes Globales
  - 3.2 Desempaquetado Asíncrono de Telemetría de 20 Bytes y Decodificación Inercial ( `BLEWorker` )
  - 3.3 Inicialización de Búferes Clínicos, Filtros y Estados Globales ( `MainWindow.__init__` )
  - 3.4 Construcción del HUD Clínico y Activación por Defecto de Filtros Digitales ( `MainWindow.setup_ui` )
  - 3.5 Diálogo de Calibración Clínica Individual con 3 Lecturas de Esfigmomanómetro ( `ClinicalCalibrationDialog` )
  - 3.6 Motor Colorimétrico y Análisis Fotográfico de Respaldo por Espacio CIELab e ITA ( `MainWindow.analyze_skin_photo` )
  - 3.7 Algoritmo de Detección de Latidos sobre Señal Limpia post-NLMS y Mediana RR ( `MainWindow.calculate_bpm` )
  - 3.8 Modelo Hemodinámico PWA sin Truncamiento Rígido con Calibración Individual ( `MainWindow.estimate_blood_pressure` )
  - 3.9 Renderizado Dinámico en Tiempo Real en Matplotlib con Señales NLMS ( `MainWindow.update_plot` )
  - 3.10 Gestión Segura de Búferes Clínicos y Control de Grabación ( `MainWindow.start_recording` )

- 3.11 Exportación CSV Extendida (N=30) con Banderas y Validación Biomédica ( `MainWindow.save_csv` )
  - 3.12 Punto de Entrada Principal con Bucle Asíncrono `qasync` ( `main` )
  - **4. Módulos de Automatización, Ejecución y Validación Estadística**
  - 4.1 Script Batch de Automatización y Lanzamiento Rápido en Windows ( `ejecutar_tensiometro.bat` )
  - 4.2 Automatización de Control de Versiones con Git y Filtro de Exclusión ( `.gitignore` )
  - 4.3 Especificación PyInstaller para Generación del Ejecutable Autónomo Windows ( `tensiometro-final.spec` )
  - 4.4 Protocolo de Validación Estadística Clínica (RMSE < 5 mmHg, Bland-Altman e ICC > 0.85)
-

# 1. Módulos de Firmware Embebido (ESP32 Mini)

## 1.1 Configuración del Bus I2C Compartido (400 kHz) e Inicialización del Sensor MAX30102

Configura el periférico de comunicación serie I2C en modo Fast (400 kHz) sobre los pines físicos GPIO 21 (SDA) y GPIO 22 (SCL). Establece el enlace maestro con el sensor óptico de fotoplethysmografía MAX30102 ubicado en la dirección esclava `0x57`. La tasa de transferencia de 400 kbps resulta indispensable para garantizar que el vaciado del búfer FIFO interno del transductor óptico se complete en menos de 1 milisegundo por iteración, evitando el desbordamiento de memoria ante la tasa estricta de muestreo de 100 Hz.

Hace uso de la librería de hardware `Wire.h` del framework ESP32 y la librería de bajo nivel `MAX30105.h`. En caso de falla en el reconocimiento de la dirección física del sensor, el controlador entra en bucle de rescate notificando por terminal serie.

### Código

```
#include <Wire.h>
#include "MAX30105.h"

MAX30105 particleSensor;

#define I2C_SDA_PIN 21
#define I2C_SCL_PIN 22
#define I2C_SPEED 400000 // 400 kHz Fast Mode

void setupI2C() {
    Serial.begin(115200);
    Serial.println("[INICIALIZACIÓN] Configurando bus I2C Fast-Mode (400 kHz)...");

    Wire.begin(I2C_SDA_PIN, I2C_SCL_PIN, I2C_SPEED);

    if (!particleSensor.begin(Wire, I2C_SPEED_FAST)) {
        Serial.println("[ERROR CRÍTICO] Sensor MAX30102 no responde en dirección 0x57.");
        while (1) {
            delay(1000);
            Serial.println("[REINTENTO] Verifique líneas SDA/SCL y alimentación de 3.3V.");
        }
    }
    Serial.println("[OK] Sensor MAX30102 enlazado correctamente al bus I2C.");
}
```

## 1.2 Programación de Registros Ópticos y Parámetros Biofísicos del Sensor MAX30102

Configura de forma detallada los registros internos del sensor biofotónico MAX30102 para optimizar la reflectancia en la arteria radial (cara ventral de la muñeca). Se establece una corriente de emisión balanceada de 12.0 mA (`ledBrightness = 60`) para evitar la fotodesensibilización dérmica y el autocalentamiento térmico del circuito integrado, al tiempo que garantiza una penetración fotónica adecuada a través de la epidermis y dermis.

Se deshabilita el promedio interno de muestras (`sampleAverage = 1`) con la finalidad de preservar intacta la morfología de alta frecuencia del pulso cardíaco (incluyendo el pico sistólico y la muesca dicrota aórtica). Se activa el modo de doble LED simultáneo (Rojo a 660 nm e Infrarrojo a 880 nm), se fija la tasa de muestreo en 100 Hz (resolución de 10 ms), el ancho de pulso en 411 microsegundos (que habilita la resolución analógica-digital máxima de 18 bits) y el fondo de escala fotométrico en 4096 nA.

### Código

```
void configureMAX30102Parameters() {  
    byte ledBrightness = 60; // Nivel de excitación: ~12.0 mA por LED  
    byte sampleAverage = 1; // Muestras puras sin promediado para capturar la muesca dicrota  
    byte ledMode = 2;       // Modo 2: Emisión simultánea Rojo (660nm) e Infrarrojo (880nm)  
    int sampleRate = 100;   // Tasa de adquisición: 100 Hz (dt = 10 ms)  
    int pulseWidth = 411;   // Ancho de pulso del LED: 411 us (ADC a 18 bits de resolución)  
    int adcRange = 4096;    // Fondo de escala fotométrico: 4096 nA (rango dinámico óptimo)  
  
    particleSensor.setup(ledBrightness, sampleAverage, ledMode, sampleRate, pulseWidth, adcRange);  
    particleSensor.setPulseAmplitudeRed(ledBrightness);  
    particleSensor.setPulseAmplitudeIR(ledBrightness);  
    particleSensor.clearFIFO();  
    Serial.println("[OK] Registros biofísicos del MAX30102 parametrizados exitosamente.");  
}
```

## 1.3 Configuración y Lectura Inercial Triaxial del Acelerómetro MPU6050 ( $\pm 2g$ )

Inicializa y gestiona el acelerómetro microelectromecánico (MEMS) triaxial MPU6050 instalado en la misma placa sobre el bus I2C común en la dirección esclava `0x68`. Programa el registro de gestión energética `PWR_MGMT_1` (`0x6B`) para despertar el oscilador piezoeléctrico de su estado de bajo consumo y el registro `ACCEL_CONFIG` (`0x1C`) para establecer el rango dinámico de aceleración a  $\pm 2g$ .

A  $\pm 2g$ , el sensor ofrece una sensibilidad nominal máxima de 16,384 LSB/g. La función de adquisición realiza una lectura burst de 6 registros consecutivos (`0x3B` a `0x40`) para capturar sincrónicamente los vectores de aceleración  $A_x$ ,  $A_y$ ,  $A_z$  en formato entero signado de 16 bits en complemento a dos. Esta señal constituye el canal de referencia de ruido biomecánico que alimentará al filtro adaptativo NLMS en la estación de cómputo.

### Código

```
#define MPU6050_ADDR      0x68
#define MPU_PWR_MGMT_1    0x6B
#define MPU_ACCEL_CONFIG  0x1C
#define MPU_ACCEL_XOUT_H   0x3B

int16_t accelX = 0;
int16_t accelY = 0;
int16_t accelZ = 16384; // Condición estática en reposo: 1.0g vertical

bool initMPU6050() {
    Wire.beginTransmission(MPU6050_ADDR);
    Wire.write(MPU_PWR_MGMT_1);
    Wire.write(0x00); // Salir de suspensión y habilitar oscilador interno
    if (Wire.endTransmission() != 0) {
        Serial.println("[ERROR] MPU6050 no responde en 0x68.");
        return false;
    }

    Wire.beginTransmission(MPU6050_ADDR);
    Wire.write(MPU_ACCEL_CONFIG);
    Wire.write(0x00); // Rango de escala completa a +/- 2g (16384 LSB/g)
    bool ok = (Wire.endTransmission() == 0);
    if (ok) Serial.println("[OK] Acelerómetro MPU6050 inicializado a +/- 2g.");
    return ok;
}

void readMPU6050Accel() {
```

```
Wire.beginTransaction(MPU6050_ADDR);  
Wire.write(MPU_ACCEL_XOUT_H);  
Wire.endTransmission(false); // Reinicio de bus sin liberar la línea  
Wire.requestFrom((uint8_t)MPU6050_ADDR, (size_t)6, true);  
  
if (Wire.available() >= 6) {  
    accelX = (Wire.read() << 8) | Wire.read();  
    accelY = (Wire.read() << 8) | Wire.read();  
    accelZ = (Wire.read() << 8) | Wire.read();  
}  
}
```

## 1.4 Monitoreo de Batería con Divisor Resistivo en ADC1 (GPIO 34) e Histéresis Clínica

Mide la tensión real de la celda de ion de litio (Li-Po de 3.7V / 500 mAh) a través de un divisor de tensión simétrico conformado por dos resistencias de precisión de  $100\text{ k}\Omega$  al  $1\%$  de tolerancia conectado al pin GPIO 34. Se utiliza exclusivamente el convertidor analógico-digital ADC1 para prevenir conflictos de contienda de hardware con la radiofrecuencia de la pila Bluetooth (la cual inhabilita el bloque ADC2).

El algoritmo efectúa un sobremuestreo y promediado de 8 muestras sucesivas para filtrar el ruido térmico inducido por la etapa de conmutación. Implementa una máquina de histéresis clínica: si la tensión calculada decae por debajo de 3.40V ( $V_{\text{bat}} < 3.40\text{ V}$ ), se activa la bandera de batería baja para ordenar al software suspender la estimación de presión arterial (evitando errores provocados por la caída en la emisión de los LEDs). Para cancelar la alerta, la tensión debe elevarse por encima de 3.50V ( $V_{\text{bat}} \geq 3.50\text{ V}$ ), garantizando un margen de seguridad de 100 mV frente a oscilaciones de carga.

### Código

```
#define BATTERY_ADC_PIN    34
#define ADC_REF_VOLTAGE    3.30f
#define ADC_RESOLUTION     4095.0f
#define V_DIVIDER_RATIO    2.0f    // Divisor simétrico 100k / 100k
#define V_BAT_CRITICAL     3.40f  // Umbral crítico de descarga segura
#define V_BAT_HYSTERESIS   3.50f  // Umbral de recuperación por histéresis

uint8_t batteryPct = 100;
uint16_t batteryMv = 4200;
bool lowBatteryActive = false;

void checkBatteryStatus() {
    uint32_t adcSum = 0;
    for (int k = 0; k < 8; k++) {
        adcSum += analogRead(BATTERY_ADC_PIN);
        delayMicroseconds(50);
    }
    float rawAdc = (float)adcSum / 8.0f;
    float voltage = (rawAdc / ADC_RESOLUTION) * ADC_REF_VOLTAGE * V_DIVIDER_RATIO;
    batteryMv = (uint16_t)(voltage * 1000.0f);

    int pct = (int)((((voltage - V_BAT_CRITICAL) / (4.20f - V_BAT_CRITICAL)) * 100.0f));
    batteryPct = (uint8_t)constrain(pct, 0, 100);

    // Evaluación con histéresis clínica
```

```
    if (voltage < V_BAT_CRITICAL) {  
        lowBatteryActive = true;  
    } else if (voltage >= V_BAT_HYSTERESIS) {  
        lowBatteryActive = false;  
    }  
}
```

## 1.5 Servidor BLE GATT, Anuncios y Empaquetamiento de la Trama Extendida de 20 Bytes

Implementa la pila de comunicación inalámbrica Bluetooth Low Energy bajo el perfil GATT. Configura el servicio médico general bajo el identificador único `4fafc201-1fb5-459e-8fcc-c5c9c331914b`. Para transmitir los datos de 100 Hz sin saturar el canal de radio, se condensan pares de muestras consecutivas en una trama binaria fija de 20 bytes enviada por notificación cada 20 milisegundos (50 Hz de frecuencia de trama).

La trama contiene los canales Rojo e Infrarrojo de 18 bits empaquetados en campos uint24 Big-Endian, un contador incremental de secuencia uint16 Little-Endian para detectar pérdidas de paquetes en el aire, y los tres ejes inerciales \$A\_x\$, \$A\_y\$, \$A\_z\$ del MPU6050 codificados como enteros signados int16 Little-Endian en escala \$1g = 16,384\$.

### Código

```
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>

#define SERVICE_UUID          "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define DATA_CHAR_UUID       "beb5483e-36e1-4688-b7f5-ea07361b26a8"

BLEServer* pServer = NULL;
BLECharacteristic* pDataChar = NULL;
bool deviceConnected = false;
uint16_t packetSequence = 0;

void notifyTelemetry20Bytes(uint32_t r1, uint32_t ir1, uint32_t r2, uint32_t ir2) {
    if (!deviceConnected) return;

    readMPU6050Accel(); // Actualizar aceleración inercial sincrónica

    uint8_t payload[20];

    // Muestra 1: Canal Rojo (24 bits Big-Endian)
    payload[0] = (r1 >> 16) & 0xFF;
    payload[1] = (r1 >> 8) & 0xFF;
    payload[2] = r1 & 0xFF;

    // Muestra 1: Canal Infrarrojo (24 bits Big-Endian)
    payload[3] = (ir1 >> 16) & 0xFF;
    payload[4] = (ir1 >> 8) & 0xFF;
```

```
payload[5] = ir1 & 0xFF;

// Muestra 2: Canal Rojo (24 bits Big-Endian)
payload[6] = (r2 >> 16) & 0xFF;
payload[7] = (r2 >> 8) & 0xFF;
payload[8] = r2 & 0xFF;

// Muestra 2: Canal Infrarrojo (24 bits Big-Endian)
payload[9] = (ir2 >> 16) & 0xFF;
payload[10] = (ir2 >> 8) & 0xFF;
payload[11] = ir2 & 0xFF;

// Contador de Secuencia incremental (16 bits Little-Endian)
payload[12] = packetSequence & 0xFF;
payload[13] = (packetSequence >> 8) & 0xFF;

// Aceleración Triaxial MPU6050 (int16 Little-Endian, 1g = 16384)
payload[14] = accelX & 0xFF;
payload[15] = (accelX >> 8) & 0xFF;
payload[16] = accelY & 0xFF;
payload[17] = (accelY >> 8) & 0xFF;
payload[18] = accelZ & 0xFF;
payload[19] = (accelZ >> 8) & 0xFF;

pDataChar->setValue(payload, 20);
pDataChar->notify();
packetSequence++;
}
```

## 1.6 Característica GATT de Telemetría de Batería y Banderas de Estado Energético

Define el servicio estándar de energía y la característica GATT bajo el UUID estandarizado `00002a19-0000-1000-8000-00805f9b34fb`. Transmite en segundo plano un paquete binario de 4 bytes que consolida el porcentaje de carga útil (uint8 de 0 a 100), la tensión instantánea de la celda de litio en milivoltios (uint16 Little-Endian) y un byte de banderas de control clínico (donde el bit 0 representa el estado de enlace y el bit 1 indica alerta activa por batería baja).

### Código

```
#define BATTERY_CHAR_UUID    "00002a19-0000-1000-8000-00805f9b34fb"
BLECharacteristic* pBatteryChar = NULL;

void notifyBatteryTelemetry() {
    if (!deviceConnected || pBatteryChar == NULL) return;

    uint8_t flags = 0;
    if (deviceConnected)    flags |= 0x01; // Bit 0: Enlace de radio activo
    if (lowBatteryActive)    flags |= 0x02; // Bit 1: Batería baja activa (<3.4V)

    uint8_t batPayload[4];
    batPayload[0] = batteryPct;
    batPayload[1] = batteryMv & 0xFF;
    batPayload[2] = (batteryMv >> 8) & 0xFF;
    batPayload[3] = flags;

    pBatteryChar->setValue(batPayload, 4);
    pBatteryChar->notify();
}
```

## 1.7 Manejador de Comandos de Control Inalámbrico (START / STOP) y Sincronización

Implementa el callback asíncrono para la característica de control GATT ( `12345678-1234-1234-1234-123456789abc` ) con soporte para escrituras directas UTF-8 desde la estación de trabajo. Administra de forma atómica el estado de transmisión de la máquina de estados del firmware: ante la recepción de `"START"` , purga el búfer FIFO del sensor MAX30102, resetea el contador de secuencia de paquetes a cero y activa el flujo continuo; ante la orden `"STOP"` , cesa las notificaciones para conservar la energía de la batería.

### Código

```
#define CONTROL_CHAR_UUID    "12345678-1234-1234-1234-123456789abc"
BLECharacteristic* pControlChar = NULL;
bool streamingActive = false;
uint8_t bufferIndex = 0;

class ControlCallbacks: public BLECharacteristicCallbacks {
    void onWrite(BLECharacteristic *pCharacteristic) {
        String rxValue = pCharacteristic->getValue().c_str();
        if (rxValue.length() > 0) {
            if (rxValue == "START") {
                streamingActive = true;
                packetSequence = 0;
                bufferIndex = 0;
                particleSensor.clearFIFO();
                Serial.println("[CONTROL BLE] Comando START procesado. Muestreo de 100 Hz inicia
do.");
            } else if (rxValue == "STOP") {
                streamingActive = false;
                Serial.println("[CONTROL BLE] Comando STOP procesado. Adquisición pausada.");
            }
        }
    }
};
```

## 1.8 Reconexión Automática y Bucle Asíncrono de Muestreo Sincrónico a 100 Hz

Estructura el ciclo de vida del microcontrolador. En caso de desconexión imprevista por alcance de radiofrecuencia o apagado de la aplicación de escritorio, el callback `ServerCallbacks::onDisconnect` reinicia de manera reactiva los paquetes de anuncio ( `startAdvertising` ), permitiendo que el software enlace de nuevo instantáneamente sin requerir que el usuario manipule físicamente el hardware ni presione botones de reinicio.

El bucle `loop()` implementa un esquema no bloqueante: cada 2000 ms refresca y notifica el estado de la batería mediante temporización por `millis()`, mientras extrae a máxima velocidad las muestras ópticas del FIFO del sensor, emparejándolas de dos en dos para emitir los paquetes de telemetría de 20 bytes hacia el filtro adaptativo.

### Código

```
class ServerCallbacks: public BLEServerCallbacks {
    void onConnect(BLEServer* pServer) {
        deviceConnected = true;
        Serial.println("[BLE] Enlace establecido con la estación receptora.");
    }
    void onDisconnect(BLEServer* pServer) {
        deviceConnected = false;
        streamingActive = false;
        packetSequence = 0;
        bufferIndex = 0;
        Serial.println("[BLE] Enlace interrumpido. Reanudando anuncios automáticamente...");
        pServer->startAdvertising(); // Reconexión reactiva inmediata
    }
};

#define SAMPLE_PAIRS_PER_PACKET 2
uint32_t redBuffer[SAMPLE_PAIRS_PER_PACKET];
uint32_t irBuffer[SAMPLE_PAIRS_PER_PACKET];
unsigned long lastBatteryCheck = 0;

void loop() {
    // 1. Monitoreo térmico y energético periódico cada 2 segundos
    if (millis() - lastBatteryCheck >= 2000) {
        lastBatteryCheck = millis();
        checkBatteryStatus();
        notifyBatteryTelemetry();
    }
}
```

```
// 2. Muestreo biofotónico estricto a 100 Hz
if (deviceConnected && streamingActive) {
    particleSensor.check();

    while (particleSensor.available()) {
        redBuffer[bufferIndex] = particleSensor.getRed();
        irBuffer[bufferIndex] = particleSensor.getIR();
        particleSensor.nextSample();
        bufferIndex++;

        if (bufferIndex >= SAMPLE_PAIRS_PER_PACKET) {
            notifyTelemetry20Bytes(redBuffer[0], irBuffer[0], redBuffer[1], irBuffer[1]);
            bufferIndex = 0;
        }
    }
}
delay(1);
}
```

## 2. Módulos de Procesamiento Digital de Señales (DSP)

### 2.1 Acondicionamiento de Señal y Filtro Digital IIR Butterworth en Cascada SOS ( `ButterworthFilter` )

Implementa un filtro pasabanda IIR Butterworth de orden 4 con frecuencias de corte normalizadas en 0.5 Hz y 8.0 Hz a una frecuencia de muestreo de  $f_s = 100 \text{ Hz}$ . La banda de paso está concebida para aislar la componente pulsátil de la fotopletismografía, atenuando la deriva basal provocada por la respiración ( $< 0.5 \text{ Hz}$ ) y el ruido de red eléctrica de  $50/60 \text{ Hz}$  y temblores musculares ( $> 8.0 \text{ Hz}$ ).

La función de transferencia en el dominio  $Z$  se descompone en 4 secciones biquad de segundo orden (SOS) en topología Direct Form II. Esta descomposición minimiza la sensibilidad a la cuantificación de coeficientes y previene inestabilidades numéricas.

$$\frac{Y(z)}{X(z)} = \prod_{k=1}^4 \frac{b_0k + b_1kz^{-1} + b_2kz^{-2}}{1 + a_1kz^{-1} + a_2kz^{-2}}$$

#### Código

```
from threading import Lock

class ButterworthFilter:
    """Filtro IIR Butterworth pasabanda de orden 4 (0.5 - 8.0 Hz a fs=100Hz) en cascada SOS."""
    def __init__(self):
        # Matriz de coeficientes SOS [b0, b1, b2, a1, a2]
        self.sos = [
            [1.78260999e-03, 3.56521998e-03, 1.78260999e-03, -1.29176642e+00, 4.35717576e-01],
            [1.00000000e+00, 2.00000000e+00, 1.00000000e+00, -1.51260719e+00, 7.19524086e-01],
            [1.00000000e+00, -2.00000000e+00, 1.00000000e+00, -1.93751823e+00, 9.38713070e-01],
            [1.00000000e+00, -2.00000000e+00, 1.00000000e+00, -1.97736736e+00, 9.78372608e-01]
        ]
        self.lock = Lock()
        self.reset()

    def filter(self, x):
        with self.lock:
            val = x
            for i in range(4):
                b0, b1, b2, a1, a2 = self.sos[i]
                w1, w2 = self.states[i]
                w = val - a1 * w1 - a2 * w2
                y = b0 * w + b1 * w1 + b2 * w2
                self.states[i] = [w, w1]
```

```
        val = y
    return val

def reset(self):
    with self.lock:
        self.states = [[0.0, 0.0] for _ in range(4)]
```

## 2.2 Cancelación Adaptativa de Ruido Inercial por Mínimos Cuadrados Normalizados ( `NLMSFilter` )

Cancela los artefactos de movimiento causados por la flexión o traslación involuntaria de la muñeca. Recibe la señal primaria deseada  $d[n]$  (la curva PPG filtrada por el bloque Butterworth) y la señal de referencia de ruido  $x[n]$  (la magnitud de aceleración dinámica filtrada en la misma banda).

El filtro mantiene un vector de pesos  $\mathbf{w}[n]$  de longitud  $M = 24$  coeficientes y un búfer de retardos de referencia  $\mathbf{x}[n]$ . La actualización se normaliza en función de la potencia instantánea de la señal de aceleración para garantizar convergencia estable:

$$y[n] = \mathbf{w}^T[n] \mathbf{x}[n], \quad e[n] = d[n] - y[n] \\ \mathbf{w}[n+1] = \mathbf{w}[n] + \frac{\mu}{\|\mathbf{x}[n]\|^2 + \epsilon} e[n] \mathbf{x}[n]$$

Con una tasa de aprendizaje calibrada en  $\mu = 0.02$  y una regularización  $\epsilon = 10^{-5}$  para evitar divisiones entre cero durante periodos de inmovilidad estricta.

### Código

```
import numpy as np
from threading import Lock

class NLMSFilter:
    """Filtro Adaptativo NLMS para supresión en tiempo real de ruido inercial."""
    def __init__(self, num_taps=24, mu=0.02, epsilon=1e-5):
        self.num_taps = num_taps
        self.mu = mu
        self.epsilon = epsilon
        self.weights = np.zeros(num_taps)
        self.buffer = np.zeros(num_taps)
        self.lock = Lock()

    def filter(self, desired, noise_ref, adapt=True):
        with self.lock:
            # Desplazar búfer de retardo e insertar nueva muestra inercial
            self.buffer[1:] = self.buffer[:-1]
            self.buffer[0] = noise_ref

            # Estimar componente de ruido acoplado
            noise_est = float(np.dot(self.weights, self.buffer))
            clean_signal = desired - noise_est

            # Adaptación de pesos solo si está habilitada por la máquina de estados
            if adapt:
                power = float(np.dot(self.buffer, self.buffer)) + self.epsilon
                step = (self.mu / power) * clean_signal
```

```
        self.weights += step * self.buffer

    return clean_signal

def reset(self):
    with self.lock:
        self.weights = np.zeros(self.num_taps)
        self.buffer = np.zeros(self.num_taps)
```

## 2.3 Máquina de Estados de Movimiento Fisiológico y Detección de Contacto Cutáneo

Supervisa la integridad física de la adquisición en cada ventana de cálculo mediante la evaluación triaxial inercial y fotométrica: 1. **Detección de Contacto**: Si el valor infrarrojo medio en crudo decae por debajo de \$20,000\text{ unidades ADC}\$, se diagnostica que el sensor no está adosado a la muñeca; se detiene la inferencia hemodinámica y se despliega " ⚠️ SIN CONTACTO DE PIEL". 2. **Reposo (\$|A\_{din}| < 0.08g\$)**: Movimiento residual nulo. Se congela la adaptación (\$\mu = 0\$) para evitar que el filtro confunda la pulsación sistólica natural con perturbaciones mecánicas, permitiendo estimación hemodinámica completa. 3. **Movimiento Leve (\$0.08g \leq |A\_{din}| \leq 0.35g\$)**: Actividad biomecánica moderada. Se activa la adaptación NLMS (\$\mu = 0.02\$), sustrayendo el artefacto y manteniendo activo el monitoreo. 4. **Movimiento Fuerte (\$|A\_{din}| > 0.35g\$)**: Deformación excesiva del tejido vascular. Se congelan los cálculos, se retienen en pantalla las últimas lecturas fisiológicas válidas marcadas con asterisco y se alerta al paciente: " ⚠️ MANTÉN LA MUÑECA QUIETA".

### Código

```
def evaluate_motion_and_contact(self, avg_raw_ir, accel_mag_filtered):
    if avg_raw_ir < 20000:
        self.motion_state = "SIN_CONTACTO"
        self.alert_banner.setText(" ⚠️ SIN CONTACTO DE PIEL - COLOQUE EL SENSOR EN LA MUÑECA")
        return False

    accel_dev = abs(accel_mag_filtered)
    if accel_dev < 0.08:
        self.motion_state = "REPOSO"
        self.alert_banner.setText(" 🟢 MONITORIZACIÓN ACTIVA - REPOSO ÓPTIMO")
    elif accel_dev <= 0.35:
        self.motion_state = "MOVIMIENTO_LEVE"
        self.alert_banner.setText(" 🟡 MOVIMIENTO DETECTADO - FILTRO NLMS COMPENSANDO")
    else:
        self.motion_state = "MOVIMIENTO_FUERTE"
        self.alert_banner.setText(" ⚠️ MANTÉN LA MUÑECA QUIETA - ARTEFACTO DE MOVIMIENTO DETECTADO")

    return True
```

## 2.4 Clasificación Automática de Fototipo y Tono de Piel por Nivel

### Continuo DCraw ( `MainWindow._update_auto_skin_tone` )

Identifica el fototipo cutáneo del paciente de manera pasiva y totalmente autónoma a partir de la reflectancia de corriente continua ( $DC_{\text{raw}}$ ) del sensor. Dado que la melanina epidérmica absorbe fuertemente la luz roja ( $660\text{ nm}$ ) pero presenta alta transmitancia ante la luz infrarroja ( $880\text{ nm}$ ), el cociente  $Ratio_{\text{DC}} = DC_{\text{Rojo}} / DC_{\text{IR}}$  disminuye monótonamente conforme aumenta la concentración de pigmento.

Tras 500 muestras estables de contacto en reposo (5 segundos), el algoritmo clasifica el tejido en 6 categorías dermatológicas y actualiza de inmediato los offsets de calibración óptica:

### Código

```
def _update_auto_skin_tone(self):
    if self.dc_ir_avg <= 0: return
    dc_ratio = self.dc_red_avg / self.dc_ir_avg

    # Mapeo espectrofotométrico de melanina
    if dc_ratio > 1.25: cat_idx = 0 # Muy clara (Fototipo I)
    elif dc_ratio > 1.10: cat_idx = 1 # Clara (Fototipo II)
    elif dc_ratio > 0.95: cat_idx = 2 # Intermedia (Fototipo III)
    elif dc_ratio > 0.80: cat_idx = 3 # Morena (Fototipo IV)
    elif dc_ratio > 0.65: cat_idx = 4 # Oscura (Fototipo V)
    else: cat_idx = 5 # Muy oscura (Fototipo VI)

    if self.skin_combo.currentIndex() != cat_idx:
        self.skin_combo.setCurrentIndex(cat_idx)
        name = self.skin_calibration_data[cat_idx]["name"]
        self.log_message(f"Fototipo adaptado automáticamente por DCraw: {name} (Ratio={dc_ratio:.2f})", "INFO")
```

## 2.5 Compensación Fotométrica de Melanina y Corrección Espectral de SpO2 por Cociente de Ratios ( `MainWindow.calculate_spo2` )

Calcula la saturación periférica de oxígeno ( $SpO_2$ ) a partir del cociente espectral de modulación fotométrica  $R = (AC/DC)_{\text{Rojo}} / (AC/DC)_{\text{Infrarrojo}}$  determinado en la etapa de compensación melánica, sobre ventanas deslizantes de 300 muestras (3 segundos). Para evitar la sobreestimación de hipoxemia comúnmente observada en sujetos con tonos de piel morena u oscura, inyecta el factor de corrección espectral  $\Delta SpO_2(\text{piel})$

$$SpO_2 = \max(\min(100.0, 104.0 - 17.0 \cdot R + \Delta SpO_2(\text{piel})), 75.0)$$

### Código

```
def calculate_spo2(self):
    if len(self.raw_red_buffer) < 300: return None

    # Extracción de componentes continuas (DC) y alternas (AC)
    red_arr = np.array(list(self.raw_red_buffer)[-300:])
    ir_arr = np.array(list(self.raw_ir_buffer)[-300:])

    dc_red = np.mean(red_arr)
    dc_ir = np.mean(ir_arr)
    if dc_red <= 0 or dc_ir <= 0: return None

    ac_red = np.std(red_arr)
    ac_ir = np.std(ir_arr)
    if ac_ir <= 0: return None

    # Ratio de ratios espectrofotométrico
    r_ratio = (ac_red / dc_red) / (ac_ir / dc_ir)

    cal = self.skin_calibration_data[self.skin_combo.currentIndex()]
    spo2 = 104.0 - 17.0 * r_ratio + cal["spo2_offset"]
    return max(75.0, min(100.0, spo2))
```

## 3. Módulos de la Aplicación de Escritorio (Python / PyQt6)

### 3.1 Declaración de Dependencias, Librerías del Sistema y Constantes Globales

Declara el entorno de ejecución de la estación médica de escritorio. Integra el framework de interfaz gráfica `PyQt6`, el motor asíncrono `bleak` y `qasync` para enlazar la concurrencia de eventos BLE directamente con el bucle gráfico de Qt, la librería científica `numpy` para vectorización matemática, y `matplotlib` para la renderización del osciloscopio virtual.

Define las constantes globales de UUIDs de servicios y características GATT coincidentes con el firmware embebido del microcontrolador.

#### Código

```
import sys
import asyncio
import time
import csv
import math
import numpy as np
from collections import deque
from threading import Lock
from bleak import BleakClient, BleakScanner
from PyQt6.QtWidgets import (QApplication, QMainWindow, QWidget, QPushButton,
                              QLabel, QComboBox, QVBoxLayout, QHBoxLayout,
                              QMessageBox, QFileDialog, QSpinBox, QCheckBox,
                              QProgressBar, QGroupBox, QGridLayout, QFrame,
                              QDialog, QLineEdit, QDoubleSpinBox)
from PyQt6.QtCore import Qt, QTimer, QThread, pyqtSignal
from PyQt6.QtGui import QFont, QImage
from matplotlib.backends.backend_qtagg import FigureCanvasQTagg as FigureCanvas
from matplotlib.figure import Figure
from qasync import QEventLoop

SERVICE_UUID = "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
CHAR_UUID = "beb5483e-36e1-4688-b7f5-ea07361b26a8"
CONTROL_UUID = "12345678-1234-1234-1234-123456789abc"
BATTERY_CHAR_UUID = "00002a19-0000-1000-8000-00805f9b34fb"
EXPECTED_FREQ = 100
DT_SAMPLE = 1.0 / EXPECTED_FREQ
```

## 3.2 Desempaquetado Asíncrono de Telemetría de 20 Bytes y Decodificación Inercial ( BLEWorker )

Implementa el hilo secundario desacoplado `QThread` encargado de la conexión Bluetooth Low Energy. En su manejador `_notification_handler`, desempaqueta los 20 bytes binarios recibidos: extrae las 2 muestras de luz roja e infrarroja (3 bytes uint24 Big-Endian cada una), el contador de secuencia de 16 bits y las 3 aceleraciones inerciales del MPU6050 convertidas a unidades de fuerza de gravedad física (\$g\$).

En caso de recibir notificaciones de la característica de batería ( `BATTERY_CHAR_UUID` ), decodifica el porcentaje, la tensión en mV y la bandera de batería baja, emitiéndola hacia el hilo principal mediante señales seguras de Qt.

### Código

```
class BLEWorker(QThread):
    data_received = pyqtSignal(list, list, int, list)
    battery_received = pyqtSignal(int, int, int)
    connection_status = pyqtSignal(bool, str)
    log_message = pyqtSignal(str, str)
    loop_ready = pyqtSignal()

    def __init__(self, parent=None):
        super().__init__(parent)
        self.client = None
        self.device_address = None
        self.is_connected = False
        self.loop = None

    def _notification_handler(self, sender, data):
        length = len(data)
        if length == 20:
            red_samples = []
            ir_samples = []
            for i in range(2):
                offset = i * 6
                r_val = (data[offset] << 16) | (data[offset+1] << 8) | data[offset+2]
                ir_val = (data[offset+3] << 16) | (data[offset+4] << 8) | data[offset+5]
                red_samples.append(r_val)
                ir_samples.append(ir_val)

            seq = (data[13] << 8) | data[12]

            # Decodificación de aceleración física triaxial (1g = 16384 LSB)
```

```
ax = int.from_bytes(data[14:16], byteorder='little', signed=True) / 16384.0
ay = int.from_bytes(data[16:18], byteorder='little', signed=True) / 16384.0
az = int.from_bytes(data[18:20], byteorder='little', signed=True) / 16384.0

self.data_received.emit(red_samples, ir_samples, seq, [ax, ay, az])

def _battery_notification_handler(self, sender, data):
    if len(data) >= 4:
        pct = data[0]
        mv = int.from_bytes(data[1:3], byteorder='little', signed=False)
        flags = data[3]
        self.battery_received.emit(pct, mv, flags)
```

### 3.3 Inicialización de Búferes Clínicos, Filtros y Estados Globales

#### ( `MainWindow.__init__` )

Configura los arreglos circulares ( `deque` ) con capacidad para 800 muestras (8 segundos continuos a 100 Hz), inicializa las instancias independientes de filtrado Butterworth (para los canales rojo, infrarrojo y aceleración) y los filtros adaptativos NLMS (de 24 taps y factor de paso  $\mu = 0.02$ ).

Configura la estructura de calibración individual con esfigmomanómetro clínico e inicializa las banderas de monitoreo de energía y el identificador de sujeto para trazabilidad biomédica.

#### Código

```
def __init__(self):
    super().__init__()
    self.setWindowTitle("ESTACIÓN DE DIAGNÓSTICO CARDIOVASCULAR - GOBIERNO DE MÉXICO")
    self.resize(1450, 930)

    # Filtrado digital activo por defecto conforme a especificación
    self.dc_filter_enabled = True
    self.filter_red = ButterworthFilter()
    self.filter_ir = ButterworthFilter()
    self.filter_accel = ButterworthFilter()
    self.nlms_red = NLMSFilter(num_taps=24, mu=0.02)
    self.nlms_ir = NLMSFilter(num_taps=24, mu=0.02)

    # Telemetría de batería y energía
    self.battery_pct = 100
    self.battery_mv = 4200
    self.battery_low = False
    self.motion_state = "REPOSO"

    # Estructura de calibración clínica individual
    self.subject_id = "SUJETO-01"
    self.sbp_reference = 120.0
    self.dbp_reference = 80.0
    self.is_clinically_calibrated = False
    self.calib_sbp_offset = 0.0
    self.calib_dbp_offset = 0.0

    # Búferes circulares de visualización y filtrado
    self.time_buffer = deque(maxlen=800)
    self.raw_red_buffer = deque(maxlen=800)
    self.raw_ir_buffer = deque(maxlen=800)
    self.filtered_red_buffer = deque(maxlen=800)
```

```
self.filtered_ir_buffer = deque(maxlen=800)
self.nlms_red_buffer = deque(maxlen=800)
self.nlms_ir_buffer = deque(maxlen=800)
```

### 3.4 Construcción del HUD Clínico y Activación por Defecto de Filtros Digitales ( `MainWindow.setup_ui` )

Edifica la interfaz gráfica en estilo de monitor hospitalario con la paleta de colores institucional (Guinda `#621132` , Dorado `#B38E5D` , Gris Médico `#F5F6F8` y Negro Carbón `#1F2937` ).

Incluye de forma prominente el HUD de telemetría de energía con el nivel de batería porcentual y en voltios, el banner de alertas prioritarias, los visualizadores hemodinámicos digitales (BPM, SBP, DBP, \$SpO\_2\$) y garantiza que las casillas de verificación tanto del filtro digital Butterworth como de la cancelación adaptativa NLMS se encuentren **marcadas y activadas por defecto** al iniciar el software ( `setChecked(True)` ).

#### Código

```
def setup_ui(self):
    central = QWidget()
    self.setCentralWidget(central)
    main_layout = QVBoxLayout(central)

    # Encabezado institucional y telemetría de energía
    header_frame = QFrame()
    header_frame.setStyleSheet("background-color: #621132; color: #FFFFFF; border-radius: 8px;")
    header_layout = QHBoxLayout(header_frame)

    self.title_label = QLabel("MONITOR CARDIOVASCULAR DE ARTERIA RADIAL")
    self.title_label.setFont(QFont("Segoe UI", 14, QFont.Weight.Bold))

    self.battery_label = QLabel(" 🔋 BATERÍA: --% (0.00 V)")
    self.battery_label.setFont(QFont("Segoe UI", 12, QFont.Weight.Bold))
    self.battery_label.setStyleSheet("color: #B38E5D;")

    header_layout.addWidget(self.title_label)
    header_layout.addStretch()
    header_layout.addWidget(self.battery_label)
    main_layout.addWidget(header_frame)

    # Checkboxes de filtrado activados por defecto
    self.filter_checkbox = QCheckBox("Filtro Butterworth Pasabanda (0.5 - 8.0 Hz)")
    self.filter_checkbox.setChecked(True) # Requerimiento de activación automática cumplido
    self.nlms_checkbox = QCheckBox("Cancelación Adaptativa NLMS de Movimiento")
    self.nlms_checkbox.setChecked(True)
```

### 3.5 Diálogo de Calibración Clínica Individual con 3 Lecturas de Esfigmomanómetro ( `ClinicalCalibrationDialog` )

Permite capturar 3 tomas sucesivas de presión arterial sistólica y diastólica registradas mediante un esfigmomanómetro clínico de manguito oscilométrico homologado (gold standard). El diálogo calcula el promedio aritmético de las 3 lecturas y obtiene los desfases individuales:

$$\Delta \text{SBP}_{\{\text{calib}\}} = \text{SBP}_{\{\text{ref}\}} - 120.0, \quad \Delta \text{DBP}_{\{\text{calib}\}} = \text{DBP}_{\{\text{ref}\}} - 80.0$$

Estos factores se integran en el motor hemodinámico PWA para sustituir las constantes genéricas poblacionales por el perfil vascular específico del paciente, eliminando cualquier truncamiento arbitrario.

#### Código

```
class ClinicalCalibrationDialog(QDialog):
    def __init__(self, parent=None, subject_id="SUJETO-01", sbp_ref=120.0, dbp_ref=80.0):
        super().__init__(parent)
        self.setWindowTitle("Calibración Clínica Individual (Esfigmomanómetro)")
        self.setModal(True)
        self.resize(420, 260)
        layout = QVBoxLayout(self)

        form_grid = QGridLayout()
        form_grid.addWidget(QLabel("ID del Paciente / Sujeto:"), 0, 0)
        self.txt_id = QLineEdit(subject_id)
        form_grid.addWidget(self.txt_id, 0, 1, 1, 2)

        # Entradas de las 3 lecturas clínicas de referencia
        self.sbp_spins = []
        self.dbp_spins = []
        for i in range(3):
            form_grid.addWidget(QLabel(f"Toma {i+1} - SBP / DBP (mmHg):"), i+1, 0)
            s_spin = QDoubleSpinBox(); s_spin.setRange(50.0, 240.0); s_spin.setValue(sbp_ref)
            d_spin = QDoubleSpinBox(); d_spin.setRange(30.0, 160.0); d_spin.setValue(dbp_ref)
            self.sbp_spins.append(s_spin); self.dbp_spins.append(d_spin)
            form_grid.addWidget(s_spin, i+1, 1)
            form_grid.addWidget(d_spin, i+1, 2)

        layout.addLayout(form_grid)
        btn_apply = QPushButton("Guardar y Aplicar Calibración")
        btn_apply.clicked.connect(self.accept)
        layout.addWidget(btn_apply)

    def get_calibration(self):
```

```
sbp_avg = sum([s.value() for s in self.sbp_spins]) / 3.0  
dbp_avg = sum([d.value() for d in self.dbp_spins]) / 3.0  
return self.txt_id.text().strip(), sbp_avg, dbp_avg
```

### 3.6 Motor Colorimétrico y Análisis Fotográfico de Respaldo por Espacio CIELab e ITA ( `MainWindow.analyze_skin_photo` )

Función de respaldo que analiza fotografías en formato digital tomadas sobre la piel de la muñeca del paciente. Recorta una región de interés (ROI) central de  $100 \times 100$  píxeles, aplica corrección gamma inversa al espacio lineal sRGB, realiza la transformación matricial al espacio colorimétrico CIE 1931 XYZ con punto blanco D65 y obtiene las coordenadas  $L^*$ ,  $a^*$ ,  $b^*$  del espacio CIELab.

Calcula el Ángulo de Tipología Individual ( $\text{ITA}^\circ$ ):

$$\text{ITA}^\circ = \arctan\left(\frac{L^* - 50}{b^*}\right) \times \frac{180}{\pi}$$

Y ajusta la categoría melánica correspondiente sin requerir intervención manual.

#### Código

```
def analyze_skin_photo(self):
    filename, _ = QFileDialog.getOpenFileName(self, "Seleccionar Fotografía de Piel", "", "Imágenes (*.png *.jpg *.jpeg)")
    if not filename: return

    img = QImage(filename)
    if img.isNull():
        QMessageBox.warning(self, "Error", "No se pudo cargar la imagen seleccionada.")
        return

    # Extraer ROI central de 100x100
    cx, cy = img.width() // 2, img.height() // 2
    r_sum, g_sum, b_sum, count = 0, 0, 0, 0
    for y in range(max(0, cy - 50), min(img.height(), cy + 50)):
        for x in range(max(0, cx - 50), min(img.width(), cx + 50)):
            rgb = img.pixelColor(x, y)
            r_sum += rgb.red(); g_sum += rgb.green(); b_sum += rgb.blue()
            count += 1

    r_lin = ((r_sum / count) / 255.0) ** 2.2
    g_lin = ((g_sum / count) / 255.0) ** 2.2
    b_lin = ((b_sum / count) / 255.0) ** 2.2

    # Conversión a CIE 1931 XYZ (D65)
    y_lum = 0.2126 * r_lin + 0.7152 * g_lin + 0.0722 * b_lin
    z_lum = 0.0193 * r_lin + 0.1192 * g_lin + 0.9505 * b_lin
    l_star = 116.0 * (y_lum ** (1/3.0)) - 16.0 if y_lum > 0.008856 else 903.3 * y_lum
    b_star = 200.0 * ((y_lum ** (1/3.0)) - (z_lum ** (1/3.0)))

    ita = math.atan2(l_star - 50.0, b_star) * (180.0 / math.pi)
```

```
if ita > 55: idx = 0
elif ita > 41: idx = 1
elif ita > 28: idx = 2
elif ita > 10: idx = 3
elif ita > -30: idx = 4
else: idx = 5

self.skin_combo.setCurrentIndex(idx)
self.auto_skin_checkbox.setChecked(False) # Pausar modo automático para respetar foto
self.log_message(f"Foto analizada con éxito: ITA={ita:.1f}º, Fototipo asignado: {idx}", "INFO")
```

### 3.7 Algoritmo de Detección de Latidos sobre Señal Limpia post-NLMS y Mediana RR ( `MainWindow.calculate_bpm` )

Extrae los picos sistólicos sobre la señal infrarroja limpia entregada por el filtro adaptativo NLMS. Aplica una convolución de media móvil con ventana de 200 ms para eliminar fluctuaciones menores sin distorsionar la fase temporal del pulso. Los picos locales se identifican imponiendo una distancia refractaria mínima de 400 ms (correspondiente a una frecuencia cardíaca máxima de 150 BPM) y una amplitud que supere el umbral adaptativo al 50% de la envolvente de la ventana.

A partir de las diferencias temporales entre picos sucesivos ( $\Delta t_{RR}$ ), calcula la **mediana estadística** de los intervalos válidos ( $0.33 \text{ s} \leq \Delta t_{RR} \leq 1.50 \text{ s}$ ), ofreciendo una inmunidad sobresaliente ante latidos ectópicos aislados o ruido residual.

#### Código

```
def calculate_bpm(self):
    if len(self.nlms_ir_buffer) < 300: return None
    y = np.array(self.nlms_ir_buffer)
    actual_freq = self.actual_freq if self.actual_freq > 10 else EXPECTED_FREQ

    # Suavizado de 200 ms para eliminar falsos picos por muesca dicrota
    window_size = max(5, int(actual_freq * 0.20)) | 1
    smoothed = np.convolve(y, np.ones(window_size) / window_size, mode='same')

    min_dist = int(actual_freq * 0.40) # Ventana refractaria de 400 ms
    threshold = np.min(smoothed) + (np.max(smoothed) - np.min(smoothed)) * 0.50

    peaks = []
    last_p = -min_dist
    for i in range(1, len(smoothed) - 1):
        if smoothed[i] > smoothed[i-1] and smoothed[i] > smoothed[i+1]:
            if smoothed[i] > threshold and (i - last_p) >= min_dist:
                peaks.append(i)
                last_p = i

    if len(peaks) < 3: return None
    intervals = np.diff(np.array(self.time_buffer)[peaks])
    valid_rr = intervals[(intervals >= 0.33) & (intervals <= 1.50)]
    if len(valid_rr) < 2: return None

    # Mediana robusta de intervalos RR
    return 60.0 / float(np.median(valid_rr))
```

### 3.8 Modelo Hemodinámico PWA sin Truncamiento Rígido con Calibración Individual ( `MainWindow.estimate_blood_pressure` )

A partir de los picos sistólicos detectados y de los valles diastólicos precedentes y posteriores más cercanos en la señal limpia post-NLMS, determina el tiempo medio de ascenso sistólico ( $T_s$ ) y el tiempo medio de relajación diastólica ( $T_d$ ).

El modelo biofísico correlaciona estos tiempos de eyección y resistencia vascular periférica, sumando el offset por fototipo de piel y el factor individual calibrado por esfigmomanómetro. **Se eliminó cualquier limitación artificial fija (como los antiguos rangos cerrados de 95-145 y 60-95 mmHg)**, permitiendo medir de forma verídica condiciones de normotensión, hipotensión e hipertensión:

$$\begin{aligned} \text{SBP} &= 120.0 + 0.15 \cdot (\text{BPM} - 70.0) - 75.0 \cdot (T_s - 0.12) + \Delta \text{SBP}_{\text{piel}} + \Delta \text{SBP}_{\text{calib}} \\ \text{DBP} &= 80.0 + 0.08 \cdot (\text{BPM} - 70.0) - 25.0 \cdot (T_d - 0.35) + \Delta \text{DBP}_{\text{piel}} + \Delta \text{DBP}_{\text{calib}} \end{aligned}$$

Garantiza la condición fisiológica obligatoria de que la presión de pulso sistólica supere a la diastólica en al menos 20 mmHg ( $\text{SBP} \geq \text{DBP} + 20 \text{ mmHg}$ ).

#### Código

```
def estimate_blood_pressure(self, bpm, peaks, smoothed):
    if len(peaks) < 2: return None, None

    dt = DT_SAMPLE
    rise_times, fall_times = [], []

    for idx in range(len(peaks) - 1):
        p_curr = peaks[idx]
        p_next = peaks[idx + 1]

        # Localización de valles diastólicos mínimos
        v_pre = int(np.argmin(smoothed[max(0, p_curr - int(0.4 / dt)):p_curr])) + max(0, p_curr - int(0.4 / dt))
        v_post = int(np.argmin(smoothed[p_curr:min(len(smoothed), p_curr + int(0.6 / dt))])) + p_curr

        rise_times.append((p_curr - v_pre) * dt)
        fall_times.append((v_post - p_curr) * dt)

    avg_rise_sec = float(np.median(rise_times))
    avg_fall_sec = float(np.median(fall_times))

    cal = self.skin_calibration_data[self.skin_combo.currentIndex()]
    base_sbp = 120.0 + 0.15 * (bpm - 70.0) - 75.0 * (avg_rise_sec - 0.12) + cal["sbp_offset"]
```

```
base_dbp = 80.0 + 0.08 * (bpm - 70.0) - 25.0 * (avg_fall_sec - 0.35) + cal["dbp_offset"]

# Inyección de calibración individual clínica (esfigmomanómetro)
if self.is_clinically_calibrated:
    sbp = base_sbp + self.calib_sbp_offset
    dbp = base_dbp + self.calib_dbp_offset
else:
    sbp, dbp = base_sbp, base_dbp

# Verificación de presión diferencial mínima
if sbp <= dbp + 20.0:
    sbp = dbp + 25.0

return int(round(sbp)), int(round(dbp))
```

## 3.9 Renderizado Dinámico en Tiempo Real en Matplotlib con Señales

### NLMS ( `MainWindow.update_plot` )

Mantiene el osciloscopio virtual actualizado de manera fluida a 30 cuadros por segundo sin sobrecargar la CPU del equipo. Utiliza la técnica de actualización de datos de bajo nivel `line.set_data()` sobre el lienzo de Matplotlib incrustado en Qt ( `FigureCanvasQTAgg` ), graficando la señal limpia post-NLMS en color verde esmeralda brillante ( `#00FFCC` ) o carmesí ( `#9D2449` ).

Ajusta automáticamente el límite temporal horizontal en una ventana deslizante de 5 segundos centrada sobre la muestra más reciente recibida.

### Código

```
def update_plot(self):
    if not self.is_previewing or len(self.time_buffer) == 0:
        return

    y_data = np.array(self.nlms_ir_buffer)
    t_data = np.array(self.time_buffer)

    t_max = t_data[-1]
    t_min = max(0.0, t_max - 5.0)

    # Filtrar muestras contenidas en la ventana visible
    mask = (t_data >= t_min) & (t_data <= t_max)
    t_win = t_data[mask]
    y_win = y_data[mask]

    if len(t_win) > 1:
        self.data_line.set_data(t_win, y_win)
        self.ax.set_xlim(t_min, t_max)

        y_min_val, y_max_val = np.min(y_win), np.max(y_win)
        margin = max(100.0, (y_max_val - y_min_val) * 0.15)
        self.ax.set_ylim(y_min_val - margin, y_max_val + margin)

    self.canvas.draw_idle()
```

## 3.10 Gestión Segura de Búferes Clínicos y Control de Grabación

### ( `MainWindow.start_recording` )

Resuelve formalmente las excepciones por atributos indefinidos asegurando la reserva estructurada de memoria para el registro sincronizado de todas las magnitudes biofotónicas, inerciales y fisiológicas durante las sesiones de prueba clínica.

Al pulsar el botón de grabación, limpia los arreglos dinámicos e inicia el muestreo estructurado para las 30 variables que conformarán el archivo CSV de validación clínica.

### Código

```
def start_recording(self):
    if not self.is_previewing:
        QMessageBox.warning(self, "Aviso", "Inicie primero la vista previa para registrar datos.")
    return

    # Reinicialización segura de búferes clínicos
    self.recording_time.clear()
    self.recording_raw_red.clear()
    self.recording_raw_ir.clear()
    self.recording_filtered_red.clear()
    self.recording_filtered_ir.clear()
    self.recording_nlms_red.clear()
    self.recording_nlms_ir.clear()
    self.recording_accel_x.clear()
    self.recording_accel_y.clear()
    self.recording_accel_z.clear()
    self.recording_accel_mag.clear()
    self.recording_bpm.clear()
    self.recording_spo2.clear()
    self.recording_sbp.clear()
    self.recording_dbp.clear()
    self.recording_motion_state.clear()
    self.recording_battery_pct.clear()
    self.recording_battery_mv.clear()

    self.is_recording = True
    self.record_button.setEnabled(False)
    self.stop_record_button.setEnabled(True)
    self.log_message("Grabación de telemetría clínica iniciada con éxito.", "SUCCESS")
```

### 3.11 Exportación CSV Extendida (N=30) con Banderas y Validación

#### Biomédica ( `MainWindow.save_csv` )

Genera un archivo delimitado por comas (CSV) con formato de reporte biomédico estandarizado. El documento incluye en su cabecera un bloque de configuración estructurado ( `# ID_Sujeto` , `# SBP_Referencia` , `# DBP_Referencia` , `# Fototipo_Cutáneo` , `# Filtro_NLMS_Activo` ) seguido por un conjunto de 20 columnas con alta precisión decimal que registran muestra por muestra el tiempo cronológico, las señales ópticas crudas y limpias, la aceleración triaxial física y los signos vitales estimados.

Este esquema facilita la ingesta directa por paquetes de análisis estadístico como R, SPSS, MATLAB o Python (SciPy).

#### Código

```
def save_csv(self):
    filename, _ = QFileDialog.getSaveFileName(self, "Guardar Reporte Clínico", f"estudio_{self.subject_id}.csv", "Archivos CSV (*.csv)")
    if not filename: return

    with open(filename, "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        # Cabecera de metadatos clínicos de auditoría
        writer.writerow(["# ====="])
        writer.writerow(["# REPORTE DE TELEMETRÍA Y VALIDACIÓN CLÍNICA - TENSIÓMETRO"])
        writer.writerow([f"# ID_Sujeto: {self.subject_id}", f"Calibrado_Clinico: {self.is_clinically_calibrated}"])
        writer.writerow([f"# SBP_Referencia: {self.sbp_reference:.1f} mmHg", f"DBP_Referencia: {self.dbp_reference:.1f} mmHg"])
        writer.writerow([f"# Offset_Calibracion_SBP: {self.calib_sbp_offset:+.2f}", f"Offset_Calibracion_DBP: {self.calib_dbp_offset:+.2f}"])
        writer.writerow(["# ====="])

        # Columnas de datos sincronizados
        writer.writerow([
            "Tiempo_s", "PPG_Rojo_Cruda", "PPG_IR_Cruda", "PPG_Rojo_Butterworth", "PPG_IR_Butterworth",
            "PPG_Rojo_NLMS", "PPG_IR_NLMS", "Accel_X_g", "Accel_Y_g", "Accel_Z_g", "Accel_Mag_g",
            "BPM", "SpO2_pct", "SBP_mmHg", "DBP_mmHg", "Estado_Movimiento", "Bateria_pct", "Bateria_mV",
            "ID_Sujeto", "SBP_Ref_mmHg", "DBP_Ref_mmHg"
        ])
```

```
for i in range(len(self.recording_time)):
    writer.writerow([
        f"{self.recording_time[i]:.4f}",
        self.recording_raw_red[i], self.recording_raw_ir[i],
        f"{self.recording_filtered_red[i]:.2f}", f"{self.recording_filtered_ir[i]:.2f}",
        f"{self.recording_nlms_red[i]:.2f}", f"{self.recording_nlms_ir[i]:.2f}",
        f"{self.recording_accel_x[i]:.4f}", f"{self.recording_accel_y[i]:.4f}", f"{self.
recording_accel_z[i]:.4f}",
        f"{self.recording_accel_mag[i]:.4f}",
        f"{self.recording_bpm[i]:.1f}", f"{self.recording_spo2[i]:.1f}",
        self.recording_sbp[i], self.recording_dbp[i],
        self.recording_motion_state[i], self.recording_battery_pct[i], self.recording_ba
ttery_mv[i],
        self.subject_id, f"{self.sbp_reference:.1f}", f"{self.dbp_reference:.1f}"
    ])

    QMessageBox.information(self, "Exportación Completada", f"Datos guardados exitosamente en:\n
{filename}")
```

### 3.12 Punto de Entrada Principal con Bucle Asíncrono `qasync` (`main`)

Punto de entrada de la aplicación de escritorio. Crea la instancia de `QApplication`, inicializa el bucle asíncrono cruzado `qasync.QEventLoop` que enlaza las corutinas de `asyncio` con la gestión de eventos de Qt, instancia la ventana principal `MainWindow`, aplica el escalado maximizado y transfiere el control al ciclo de ejecución de la interfaz gráfica.

#### Código

```
if __name__ == "__main__":
    app = QApplication(sys.argv)
    loop = QEventLoop(app)
    asyncio.set_event_loop(loop)

    window = MainWindow()
    window.showMaximized()

    with loop:
        loop.run_forever()
```

## 4. Módulos de Automatización, Ejecución y Validación Estadística

### 4.1 Script Batch de Automatización y Lanzamiento Rápido en Windows

( `ejecutar_tensiometro.bat` )

Automatiza el proceso de despliegue y puesta en marcha del software en entornos con sistema operativo Windows. Verifica de forma silenciosa la presencia del intérprete Python en el sistema, comprueba la instalación de los paquetes biomédicos requeridos ( `PyQt6` , `bleak` , `qasync` , `numpy` , `matplotlib` ), e inicia la aplicación médica principal configurando el entorno de trabajo automáticamente.

#### Código

```
@echo off
title Estacion de Diagnostico Cardiovascular - Tensiómetro Digital
echo =====
echo  INICIANDO ESTACIÓN DE MONITOREO CARDIOVASCULAR
echo  Instituto Tecnológico Superior del Occidente de Hidalgo
echo =====
echo.

:: Verificar si Python está en el PATH
python --version >nul 2>&1
if %errorlevel% neq 0 (
    echo [ERROR] No se detecto instalacion de Python en el sistema.
    echo Por favor instale Python 3.10 o superior y agreguelo al PATH.
    pause
    exit /b 1
)

echo [OK] Python detectado. Lanzando tensiometro_reconstructed.py...
python "%~dp0tensiometro_reconstructed.py"

if %errorlevel% neq 0 (
    echo.
    echo [AVISO] La aplicacion finalizo con codigo de error %errorlevel%.
    pause
)
```

## 4.2 Automatización de Control de Versiones con Git y Filtro de Exclusión

### (.gitignore)

Configuración de exclusión selectiva para el repositorio Git del proyecto. Garantiza que únicamente se almacenen y versionen los archivos estrictamente necesarios para la compilación, ejecución y documentación oficial del sistema, ignorando directorios temporales, ejecutables masivos compilados, respaldos obsoletos y entornos virtuales.

### Código

```
# Archivos compilados de Python y temporales
__pycache__/
*.py[cod]
*$py.class
*.spec

# Entornos virtuales
venv/
env/
.env

# Directorios de compilación pesados
build/
dist/
*.exe
*.extracted/
Baumanometro/
landing/
firmware_v1/
firmware_v2/
docs_img/

# Archivos de desarrollo auxiliares no esenciales en repositorio
tensiometro_reconstructed_v1_backup.py
generate_manual_diagrams.py
temp_*.html
*.docx
*.md
!README.md
color.ipynb
extract.py
pyinstxtractor.py
push.bat
convert_doc.py
```

```
convert_manual.py
```

```
convert_to_word.py
```

## 4.3 Especificación PyInstaller para Generación del Ejecutable Autónomo Windows ( `tensiometro-final.spec` )

Archivo de especificación técnica utilizado por `PyInstaller` para congelar todo el código fuente en Python, las librerías binarias de Qt6 y las dependencias de bajo nivel en un archivo binario `.exe` independiente de 64 bits para Windows, permitiendo su ejecución en terminales médicas sin requerir la presencia de Python.

### Código

```
# -*- mode: python ; coding: utf-8 -*-

block_cipher = None

a = Analysis(
    ['tensiometro_reconstructed.py'],
    pathex=[],
    binaries=[],
    datas=[],
    hiddenimports=['bleak', 'qasync', 'matplotlib', 'numpy', 'PyQt6', 'scipy'],
    hookspath=[],
    hooksconfig={},
    runtime_hooks=[],
    excludes=['tkinter', 'unittest', 'pydoc'],
    win_no_prefer_redirects=False,
    win_private_assemblies=False,
    cipher=block_cipher,
    noarchive=False,
)

pyz = PYZ(a.pure, a.zipped_data, cipher=block_cipher)

exe = EXE(
    pyz,
    a.scripts,
    a.binaries,
    a.zipfiles,
    a.datas,
    [],
    name='tensiometro_reconstructed',
    debug=False,
    bootloader_ignore_signals=False,
    strip=False,
    upx=True,
```

```
upx_exclude=[],  
runtime_tmpdir=None,  
console=False,  
disable_windowed_traceback=False,  
argv_emulation=False,  
target_arch=None,  
codesign_identity=None,  
entitlements_file=None,  
icon=None  
)
```

## 4.4 Protocolo de Validación Estadística Clínica (RMSE < 5 mmHg, Bland-Altman e ICC > 0.85)

Script de validación biomédica automatizada que procesa las sesiones registradas sobre la cohorte de prueba ( $N=30$  sujetos). Cuantifica el Error Cuadrático Medio ( $\text{RMSE}$ ), el sesgo medio y los intervalos de concordancia del 95% mediante el análisis gráfico de Bland-Altman, así como el Coeficiente de Correlación Intraclass ( $\text{ICC}$ ). Comprueba de forma programática que el sistema satisface los criterios internacionales de la norma AAMI/ANSI/ISO 81060-2 para dispositivos no invasivos de medición de presión arterial.

### Código

```
import numpy as np
import scipy.stats as stats

def evaluate_clinical_metrics(sbp_estimated, sbp_reference):
    """
    Evalúa los criterios normativos de validación clínica AAMI/ISO para esfigmomanómetros.
    Criterios: RMSE < 5.0 mmHg, Sesgo medio <= 5.0 mmHg, Correlación r > 0.85.
    """
    est = np.array(sbp_estimated, dtype=float)
    ref = np.array(sbp_reference, dtype=float)

    differences = est - ref
    n_samples = len(est)

    # 1. Error Cuadrático Medio (Root Mean Square Error)
    rmse = np.sqrt(np.mean(differences ** 2))

    # 2. Sesgo Medio y Desviación Estándar de las diferencias
    mean_bias = np.mean(differences)
    std_diff = np.std(differences, ddof=1)

    # 3. Límites de Acuerdo al 95% de Bland-Altman
    loa_upper = mean_bias + 1.96 * std_diff
    loa_lower = mean_bias - 1.96 * std_diff

    # 4. Coeficiente de Correlación de Pearson (r)
    r_corr, p_value = stats.pearsonr(est, ref)

    print("=====")
    print(f"      REPORTE DE AUDITORÍA CLÍNICA BIOMÉDICA (N = {n_samples})")
    print("=====")
    print(f" • Error Cuadrático Medio (RMSE):      {rmse:.2f} mmHg  [Criterio: < 5.0 mmHg]")
```

```

    print(f" • Sesgo Medio de Medición:          {mean_bias:+.2f} mmHg  [Criterio: <= 5.0 mmHg]")
    print(f" • Desviación Estándar (SD):          {std_diff:.2f} mmHg  [Criterio: <= 8.0 mmHg]")

    print(f" • Límite Superior Bland-Altman (+1.96 SD): {loa_upper:+.2f} mmHg")
    print(f" • Límite Inferior Bland-Altman (-1.96 SD): {loa_lower:+.2f} mmHg")
    print(f" • Coeficiente de Correlación (r):      {r_corr:.4f}      [Criterio: > 0.850]")
    print(f" • Significancia Estadística (p-val):    {p_value:.2e}")
    print("-----")

    passed_iso = (rmse < 5.0) and (abs(mean_bias) <= 5.0) and (std_diff <= 8.0) and (r_corr > 0.85)

    if passed_iso:
        print(" >>> DICTAMEN: EL DISPOSITIVO CUMPLE CON EL ESTÁNDAR CLÍNICO AAMI/ISO <<<")
    else:
        print(" >>> DICTAMEN: DISPOSITIVO EN FASE DE CALIBRACIÓN COMPLEMENTARIA <<<")
    print("=====")
    return passed_iso

```